

# PHASE 2-REQUIREMENT ANALYSIS

## Project Title: POCKETSMART AI

### Your Smart Budget & Recommendation Assistant

**Name:** Government Arts and Science College, Manur.

**College Code:** MSU142

**Team ID:** 6ab21524d329d50f0db6120b

**Team Leader:** Esakki Aruna S      **NM ID:** 55055706AAD495BADB04C8C4D3B478B8

**Team Member:** Utchi Makali      **NM ID:** 0BBEDB52C642EC0B0BA4CB4B22DE9E2D

**Team Member:** Kombu Madathi      **NM ID:** 86EE18ABFB36AEF83784FA01234C5948

**Team Member:** S Ulagammal      **NM ID:** B59DED29E07C4F78BCF442535560F984

#### 1. Introduction

PocketSmart AI is a smart budgeting and recommendation assistant intended to help users manage their personal finances. The Requirement Analysis Phase identifies what the system should do, who will use it, what information it requires, and the functional and non-functional requirements needed to develop the application.

#### 2. Purpose of the System

The purpose of PocketSmart AI is to provide a simple platform for recording income and expenses, setting budgets and savings goals, analyzing spending patterns, and generating personalized recommendations. The system should make financial information easy to understand and support better day-to-day budgeting decisions.

#### 3. Scope of the Project

- User registration and secure login.
- Recording and managing income and expenses.
- Categorizing transactions.

- Creating and monitoring monthly budgets.
- Setting and tracking savings goals.
- Analyzing spending patterns.
- Generating AI-assisted budget and purchase recommendations.
- Displaying financial summaries, charts, and alerts.
- Generating monthly spending reports.

#### **4. Stakeholders**

- End Users – Individuals who use the application to manage their personal budgets.
- Project Team – Developers and designers responsible for building and maintaining the system.
- Project Guide/Faculty – Reviews the project requirements, implementation, and outcomes.
- System Administrator – Manages system-level settings and user-related administration where required.

#### **5. Functional Requirements**

**FR-01: User Registration** – The system shall allow new users to create an account using required basic details.

**FR-02: User Login** – The system shall authenticate registered users before providing access to personal financial data.

**FR-03: Income Management** – The system shall allow users to add, edit, view, and delete income records.

**FR-04: Expense Management** – The system shall allow users to add, edit, view, and delete expense records.

**FR-05: Expense Categories** – The system shall organize expenses into categories such as food, travel, education, shopping, and entertainment.

**FR-06: Budget Management** – The system shall allow users to create category-wise and monthly budgets.

**FR-07: Budget Monitoring** – The system shall calculate spending against the selected budget and indicate remaining amounts.

**FR-08: Savings Goals** – The system shall allow users to create savings targets and monitor their progress.

**FR-09: Spending Analysis** – The system shall analyze recorded transactions and identify major spending categories and trends.

**FR-10: AI Recommendations** – The system shall provide recommendations based on the user's budget, spending patterns, and goals.

**FR-11: Purchase Assistance** – The system shall allow users to enter a planned purchase and receive a budget-based recommendation.

**FR-12: Alerts** – The system shall provide alerts when spending approaches or exceeds a defined budget limit.

**FR-13: Dashboard** – The system shall display income, expenses, remaining budget, savings progress, and relevant summaries.

**FR-14: Reports** – The system shall generate monthly or period-based summaries of income, expenses, and savings.

## **6. Non-Functional Requirements**

- **Usability:** The interface should be simple, clear, and easy for beginners to understand.
- **Performance:** Common operations such as adding expenses and viewing the dashboard should respond quickly.
- **Security:** User accounts and financial information should be protected through appropriate authentication and access controls.
- **Reliability:** The system should store transaction data accurately and minimize data loss.
- **Scalability:** The design should allow additional users and features to be added later.
- **Maintainability:** The software should use a modular structure so that components can be updated independently.
- **Availability:** The application should be accessible whenever users need to manage their budgets, subject to system availability.
- **Privacy:** Personal financial information should be handled responsibly and only used for intended application functions.

## 7. User Requirements

- Users should be able to enter income and expenses without complex steps.
- Users should be able to understand their current budget status at a glance.
- Users should be able to set realistic savings goals.
- Users should receive understandable recommendations rather than complicated financial terminology.
- Users should be able to review previous transactions and spending trends.
- Users should be able to modify or remove incorrect transaction entries.

## 8. System Inputs

- User account information.
- Income amount and source.
- Expense amount, date, and category.
- Monthly or category budget limits.
- Savings target and target date, if applicable.
- Planned purchase details.
- User preferences relevant to recommendations.

## 9. System Outputs

- Current balance and remaining budget.
- Category-wise spending summaries.
- Budget status and alerts.
- Savings progress.
- Spending trends and reports.
- AI-generated budgeting and purchase recommendations.

## 10. Basic Use Cases

**Use Case 1 – Manage Expenses** – User logs in, enters an expense, selects a category, and saves the transaction.

**Use Case 2 – Create Budget** – User selects a period and categories, enters budget limits, and saves the budget.

**Use Case 3 – Check Budget Status** – User opens the dashboard to view spending and remaining budget.

**Use Case 4 – Set Savings Goal** – User enters a savings target and tracks progress over time.

**Use Case 5 – Get Recommendation** – User provides financial details or a planned purchase, and the system generates a personalized recommendation.

**Use Case 6 – View Report** – User selects a period and views a summary of income, expenses, savings, and spending patterns.

## **11. Proposed Technology Requirements**

- Frontend: HTML, CSS, JavaScript or an appropriate web/mobile framework.
- Backend: Python using Flask, Django, or another suitable backend framework.
- Database: MySQL, SQLite, or another relational database.
- AI/Recommendation Component: Python-based recommendation logic and/or an AI service.
- Visualization: Chart.js or another suitable charting library.

## **12. Constraints and Assumptions**

- The accuracy of recommendations depends on the quality and completeness of user-provided data.
- The initial version may use manually entered transactions rather than direct bank integration.
- AI recommendations are intended as budgeting assistance and should not be treated as professional financial advice.
- The system requires appropriate authentication to protect personal financial information.

## **13. Acceptance Criteria**

- Users can successfully register and log in.
- Income and expense records can be added, edited, viewed, and deleted.
- Budgets can be created and their status can be monitored.
- Savings goals can be created and progress can be displayed.
- The dashboard accurately summarizes stored financial data.
- The recommendation module produces understandable suggestions based on available user data.

- The system provides appropriate alerts for budget limits.
- The application stores and retrieves user data reliably.

#### **14. Requirement Analysis Summary**

The requirement analysis establishes PocketSmart AI as a user-friendly budgeting assistant with four main areas: financial data management, budget and savings tracking, spending analysis, and AI-assisted recommendations. These requirements provide the foundation for the system design, database design, implementation, testing, and future enhancement phases.